<table>
<tr><td colspan="4">Name of Student: Mukteshwar Bhupendra Joshi</td></tr>
<tr><td colspan="2">Roll Number: 24/B</td><td colspan="2">Distributed System and Cloud Computing</td></tr>
<tr><td colspan="4">Practical 10: Develop application for amazon aws using amazon aws platform training kit and visual studio</td></tr>
<tr><td colspan="2">DOP: 06/10/2025</td><td colspan="2">DOS: 13/10/2025</td></tr>
<tr><td>CO Mapped:
CO5</td><td>PO Mapped:
PO1,PO2,PO4,PO7,PSO1,PSO2</td><td>Faculty Signature:</td><td>Marks:</td></tr>
</table>

PRACTICAL NO. 10

Aim

To develop Application for Windows Azure / Amazon AWS using Windows Azure Platform Training Kit and Visual Studio.

Descriptions:
Developing Applications for Windows Azure / Amazon AWS using Windows Azure Platform Training Kit and Visual Studio
Introduction
Cloud computing has transformed the way applications are developed, deployed, and managed. Two of the most popular cloud platforms available today are Microsoft Azure (formerly Windows Azure) and Amazon Web Services (AWS). Both platforms provide a wide range of services such as computing power, storage, databases, networking, and deployment tools that enable developers to build scalable and reliable applications.
Developing applications for Azure or AWS can be easily done using Visual Studio, a powerful Integrated Development Environment (IDE) from Microsoft. Visual Studio provides built-in tools, extensions, and templates for creating, testing, and deploying cloud-based applications seamlessly. Additionally, the Windows Azure Platform Training Kit offers tutorials, hands-on labs, and sample applications that help developers understand and practice cloud development.

1. Understanding Windows Azure Platform Training Kit
The Windows Azure Platform Training Kit is a comprehensive resource provided by Microsoft to help developers learn and build cloud applications using Azure. It contains:

- Hands-on Labs – Step-by-step exercises to guide users through real Azure development scenarios.
- Presentations and Demos – Detailed presentations on cloud computing concepts, Azure services, and best practices.
- Sample Applications – Example projects that demonstrate how to use Azure services such as Blob Storage, Queues, Tables, and App Services.

Using this kit, developers can practice deploying web applications, configuring virtual machines, managing storage, and implementing security and scalability features in Azure.

2. Role of Visual Studio in Cloud Application Development
Visual Studio plays a central role in developing and deploying applications on both Azure and AWS. It supports multiple programming languages like C#, Java, Python, and Node.js and integrates cloud SDKs for easier management.
Key Features:

- Azure SDK Integration: Visual Studio provides templates for creating Azure Web Apps, Functions, and Cloud Services.
- AWS Toolkit for Visual Studio: This toolkit allows developers to easily connect to AWS, deploy applications, and manage resources like EC2, S3, and Lambda directly from the IDE.
- Debugging and Testing: Developers can run cloud applications locally using emulators before deployment.
- Continuous Deployment: Visual Studio integrates with GitHub and Azure DevOps for CI/CD pipelines, enabling automated testing and deployment.

3. Developing an Application for Azure

Steps:

1. Install Azure SDK for Visual Studio – This adds Azure templates and tools to your IDE.
2. Create a New Project – Open Visual Studio → Create a new project → Select "Azure Cloud Service" or "ASP.NET Web App".
3. Add Azure Services – Configure storage, databases (like Azure SQL), and authentication (Azure AD).
4. Test Locally – Use the local Azure Emulator to test your application before deployment.
5. Deploy to Azure – Right-click the project → "Publish" → Select your Azure subscription and deploy the app.

Example:

A developer can build a web application using ASP.NET Core, connect it to an Azure SQL Database, and host it on Azure App Service with automatic scaling and monitoring.

4. Developing an Application for AWS

Steps:

1. Install AWS Toolkit for Visual Studio – This extension adds AWS templates and integration options.
2. Sign in to AWS Account – Configure AWS credentials (Access Key and Secret Key) within Visual Studio.
3. Create Project – Choose a template such as "AWS Serverless Application" or "Elastic Beanstalk Application."
4. Build and Test Locally – Run the app and test AWS services integration like DynamoDB, S3, or Lambda.
5. Deploy to AWS – Right-click → "Publish to AWS" → Select service (EC2, Elastic Beanstalk, Lambda) and deploy.

Example:

You can create a C# ASP.NET Core web application that uploads files to Amazon S3 and stores metadata in Amazon DynamoDB using AWS SDK for .NET.

5. Benefits of Using Azure/AWS with Visual Studio

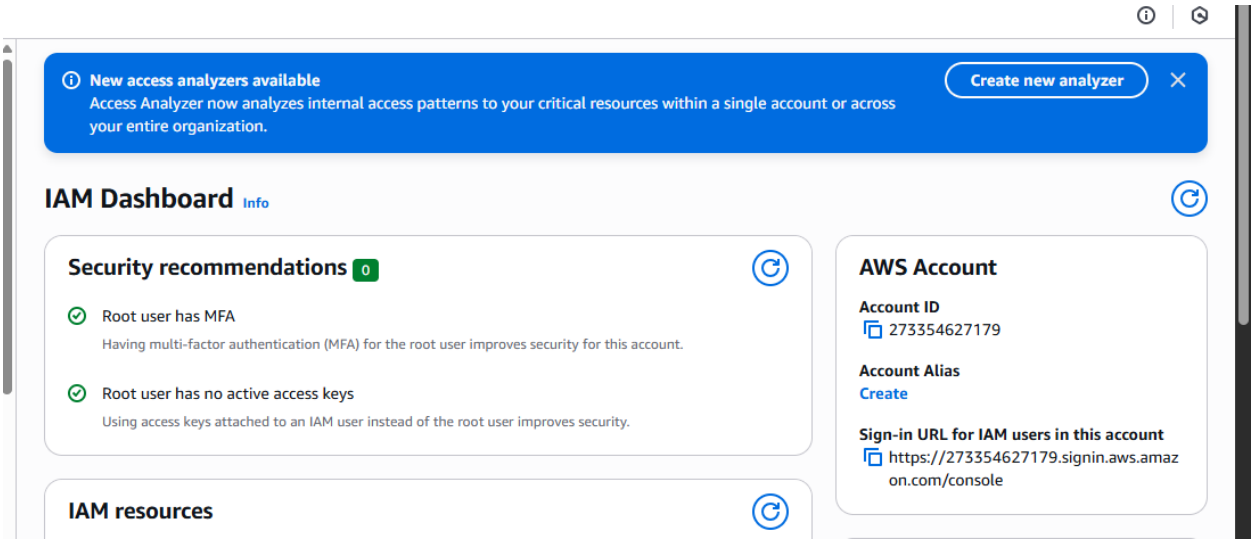
- Ease of Development: Prebuilt templates and SDK integration simplify coding and deployment.
- Scalability: Applications can automatically scale up or down depending on usage.
- Reliability: Cloud platforms offer redundancy and backup options to ensure high availability.
- Security: Integration with IAM (AWS) and Azure AD ensures secure access control.
- Cross-Platform Support: Developers can deploy .NET, Java, Node.js, or Python applications easily.

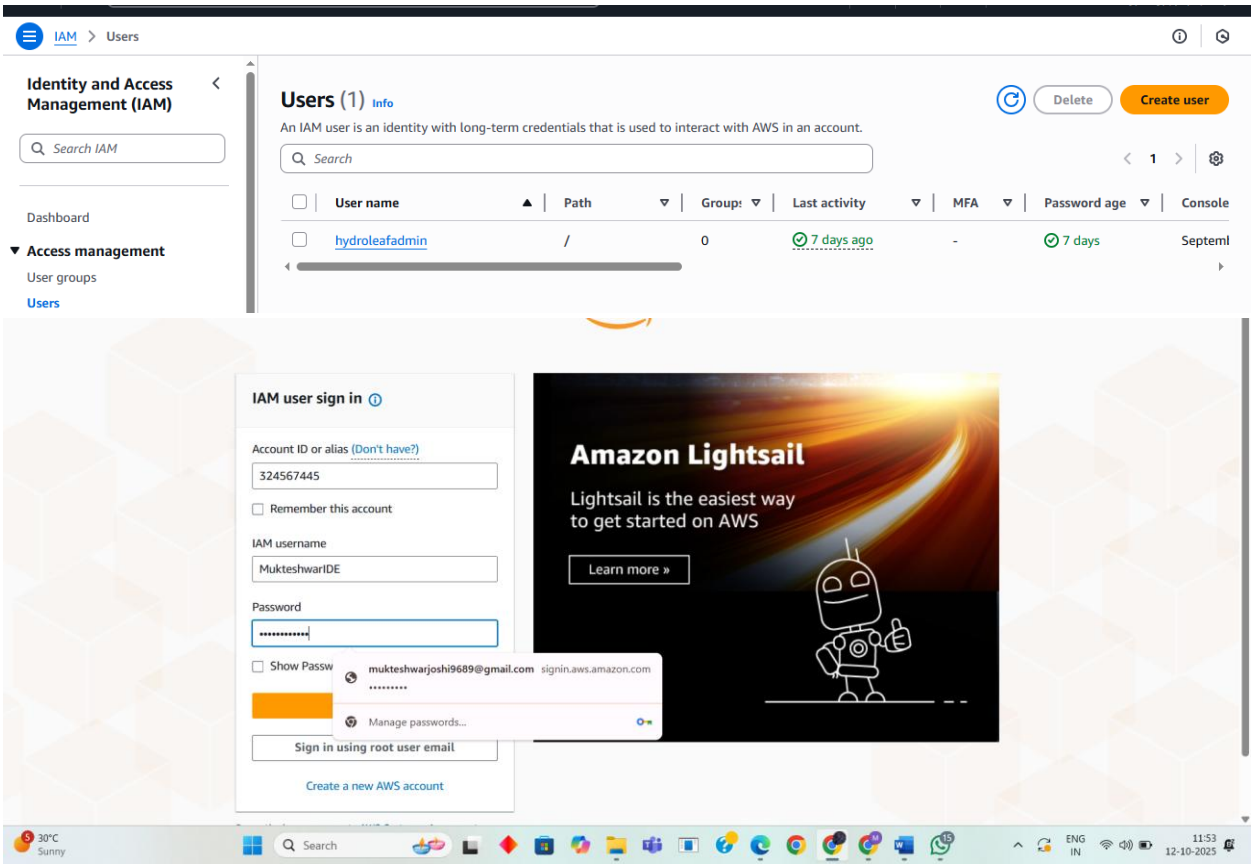
Tools and Technologies Used

- Amazon Web Services (AWS): Elastic Beanstalk, EC2, S3, RDS (PostgreSQL), Load Balancer
- Programming Environment: Visual Studio
- Database: Amazon RDS (PostgreSQL) for deployment and SQL Lite for developing
- Storage: Amazon S3 (Simple Storage Service)
- Deployment Environment: AWS Elastic Beanstalk

Procedure:

Login to the IAM user before deploying the application , never deploy on the root user account.

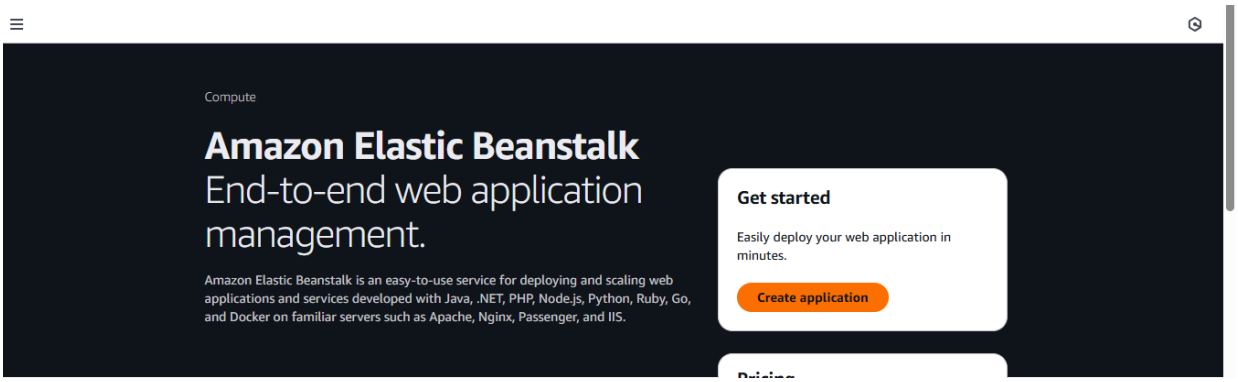




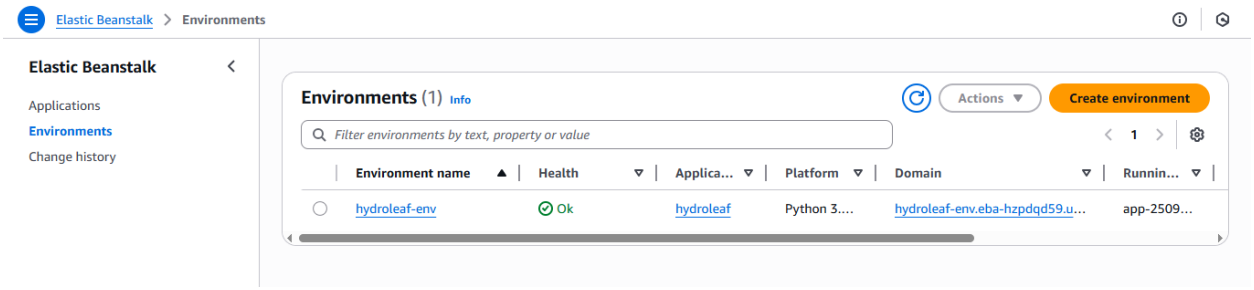
1. **Application Development**
   Designed and developed a web application using Visual Studio. Application logic implemented for user interaction and data storage.

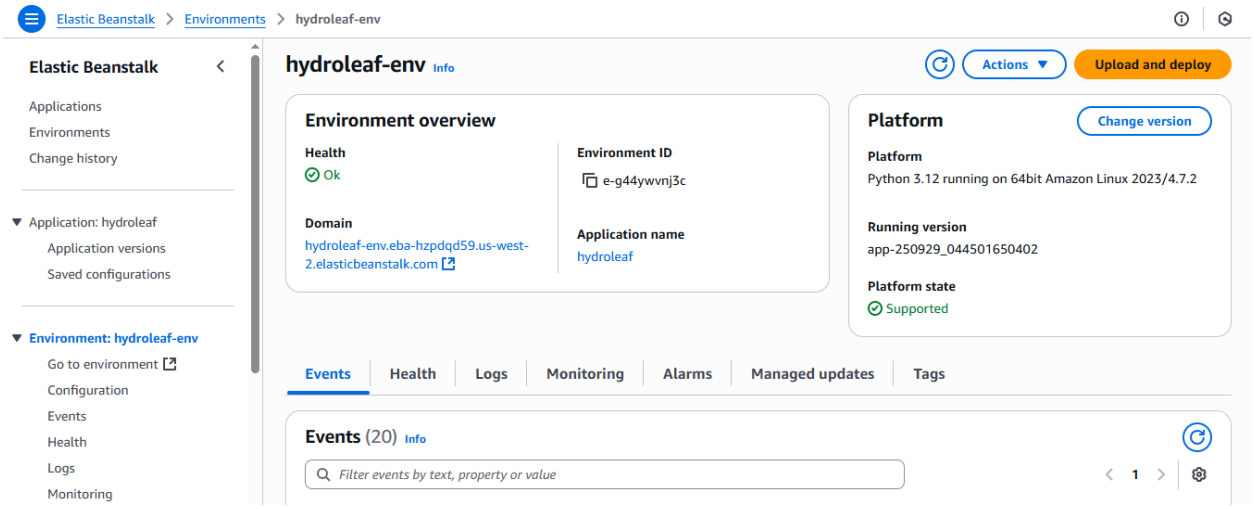
2. **Deployment using Elastic Beanstalk**
   Created an Elastic Beanstalk environment on AWS.

Deployed the developed web application directly to Elastic Beanstalk. Elastic Beanstalk managed provisioning, application deployment, and scaling.

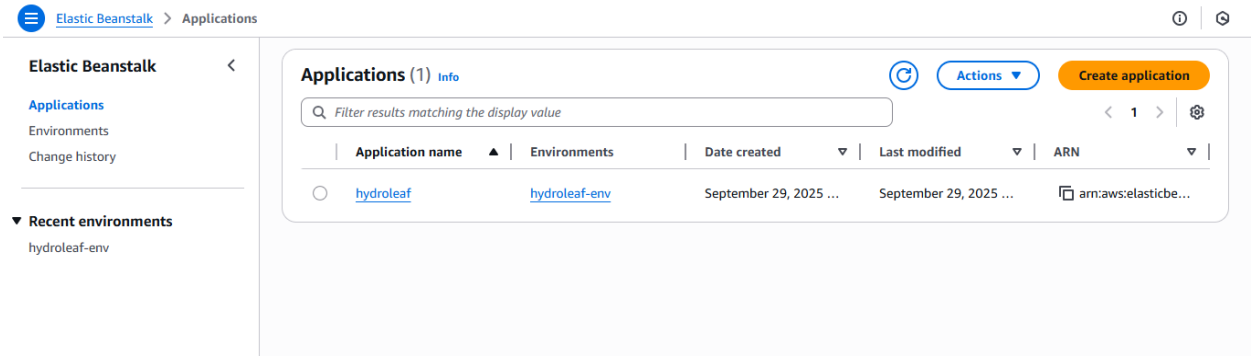


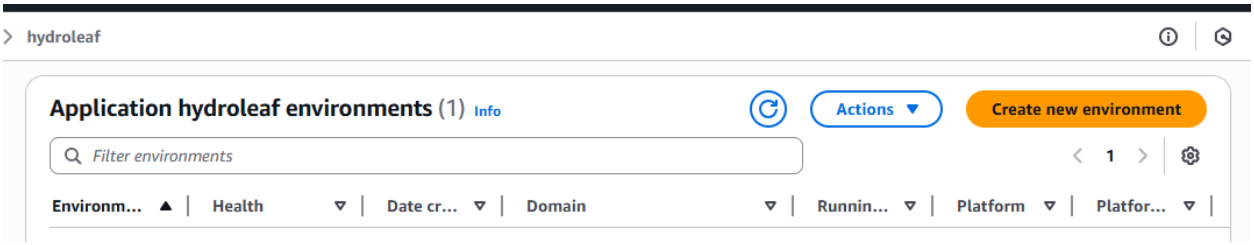
Elastic bean environment to run the website/Application



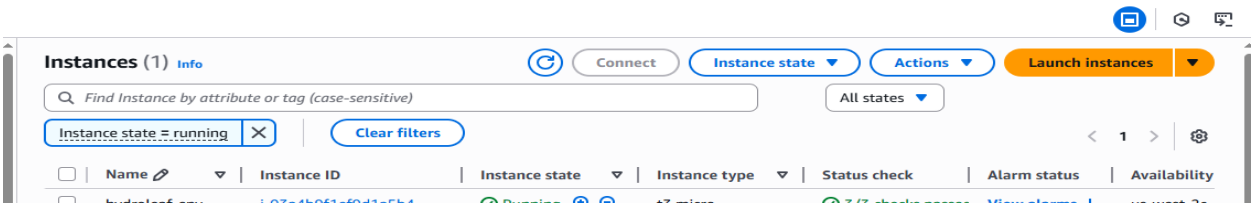


Application deployed on Elastic bean server:





3. **Server Hosting with EC2 Instance**
   Elastic Beanstalk automatically launched EC2 instances for hosting the application server.
   Configured EC2 instance security groups for HTTP/HTTPS access.

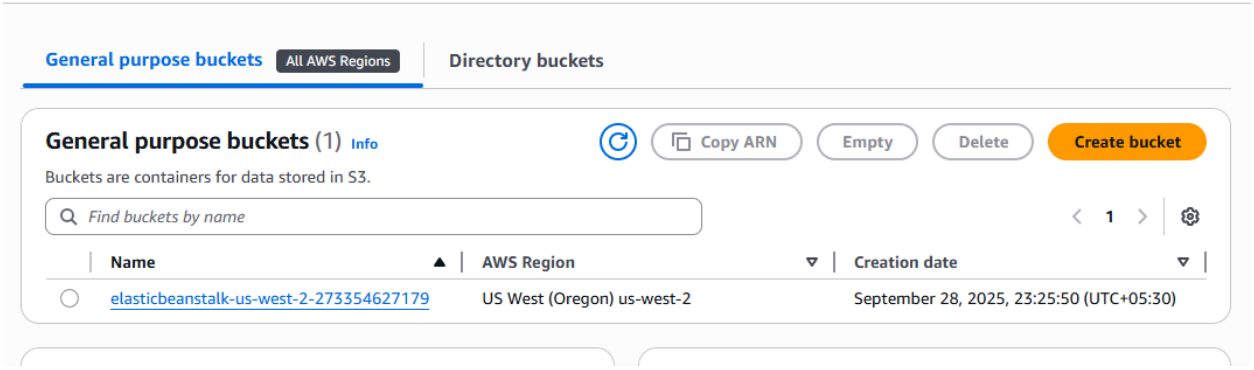


Note: we are running on t3.micro server which comes under free tier 720 hours per month

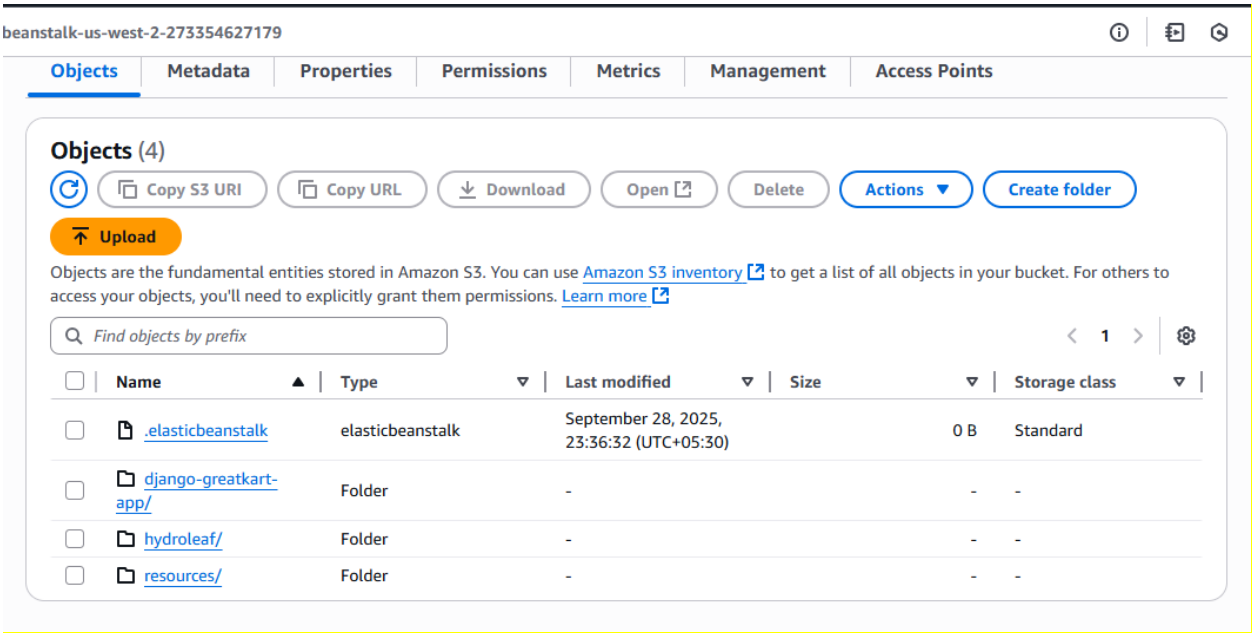
4. **Database Integration**
   Configured **Amazon RDS PostgreSQL** as the backend database. Established secure connectivity between EC2 instance (application) and RDS.

5. **Data Storage with S3**
   Used **Amazon S3 buckets** for storing static data, media files, and backups.

Integrated application with S3 via AWS SDK for data retrieval and upload.



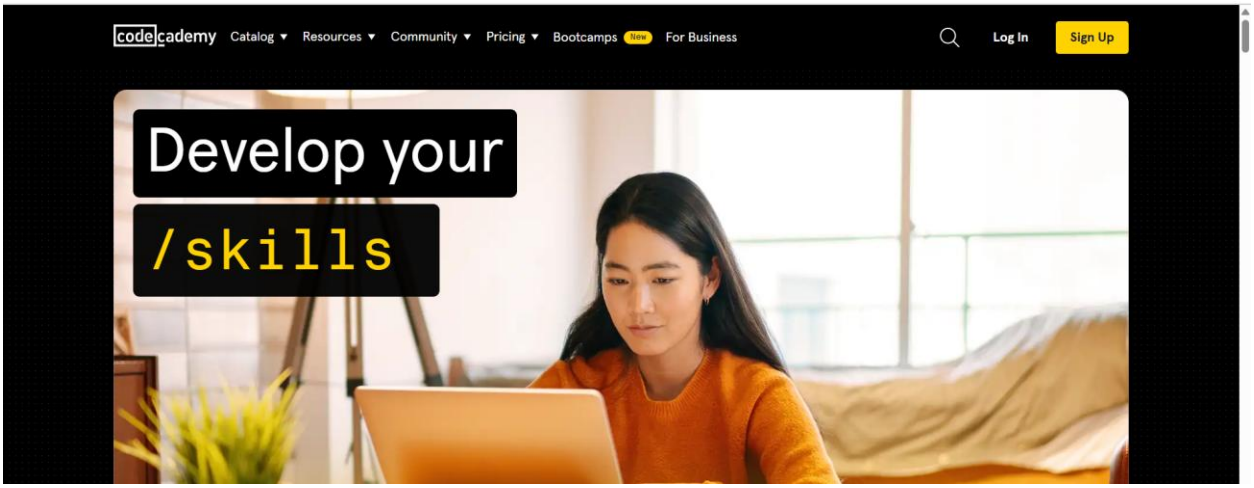
This contain all the static files for the deployed Application / Website



6. **Load Balancing**
   Configured **Elastic Load Balancer** to distribute incoming traffic across multiple EC2 instances.
   Ensured fault tolerance, high availability, and scalability of the application.

Output:

A fully functional cloud-based web application was successfully developed and deployed on AWS using Elastic Beanstalk. The system integrated with **EC2 for hosting, RDS for database management, S3 for storage, and Load Balancers for scalability and availability.**



Conclusion

This practical demonstration demonstrated how to design, deploy, and manage a distributed application on AWS. By leveraging cloud services such as Elastic Beanstalk, EC2, RDS, S3, and Load Balancing, the application achieved scalability, reliability, and high availability, showcasing the power of cloud computing platforms.